Indian Institute of Technology Mandi

School of Computing and Electrical Engineering

Fake Image Detection

A Dual-Branch Deep Learning Pipeline using Self-Supervised Learning and Frequency-Domain Analysis

Technical Research Report

Institution:	Indian Institute of Technology Mandi
Domain:	Computer Vision / Media Forensics
Framework:	PyTorch 2.x
Primary Model:	DINO-ViT-B/16 + ResNet-18 (FFT)
Dataset:	FaceForensics++ (FF++)

This report presents a complete, reproducible pipeline for detecting AI-generated and manipulated face images. It covers theoretical foundations, architectural decisions, mathematical derivations, training protocol, and full PyTorch implementation.

March 31, 2026

Contents

1 Introduction and Problem Statement **3**
1.1 Why Is Detection Hard? . . . 3
1.2 Our Approach: A Dual-Branch Architecture . . . 3

2 Dataset: FaceForensics++ (FF++) **5**
2.1 Why FaceForensics++? . . . 5
2.2 Dataset Structure . . . 5
2.3 Dataset Split and Your Custom Folder . . . 5
2.4 Data Pre-extraction from FF++ Videos . . . 6

3 Stage 1: Face Detection and Alignment **7**
3.1 Why Alignment Is Necessary . . . 7
3.2 Face Detection: RetinaFace . . . 7
3.3 Mathematical Formulation of Alignment . . . 7
3.4 Output . . . 8

4 Stage 2: Preprocessing and Data Augmentation **9**
4.1 Standard Preprocessing . . . 9
4.2 Data Augmentation Strategy . . . 9
4.2.1 Regime A — Spatial Augmentations (Applied to Both Branches) . 9
4.2.2 Regime B — Compression Augmentations (Critical for Real-World Robustness) . . . 9

5 Stage 3: The Spatial Branch — DINO-Pretrained ViT-B/16 **10**
5.1 Why a Vision Transformer? . . . 10
5.2 ViT-B/16 Architecture . . . 10
5.2.1 Patch Embedding . . . 10
5.2.2 Multi-Head Self-Attention (MHSA) . . . 10
5.2.3 Output Features . . . 11
5.3 Why DINO Pretraining? . . . 11
5.3.1 DINO: Self-Distillation with No Labels . . . 11
5.4 Pretrained Weights: What We Use . . . 12

6 Stage 4: The Frequency Branch — FFT + ResNet-18 **13**
6.1 Why Frequency Analysis? . . . 13
6.2 2-D Discrete Fourier Transform (DFT) . . . 13
6.3 Our Frequency Representation . . . 13
6.4 CNN Encoder: ResNet-18 . . . 14

7 Stage 5: Cross-Attention Fusion **15**
7.1 Why Not Simple Concatenation? . . . 15
7.2 Cross-Attention Mechanism . . . 15

8 Stage 6: Classification Head and Loss Function **17**
8.1 Classification Head Architecture . . . 17
8.2 Loss Function: Binary Cross-Entropy with Label Smoothing . . . 17
8.3 Class Imbalance Handling . . . 17

9 Complete Pipeline Architecture **19**
9.1 Full Mathematical Flow . . . 19
9.2 Architecture Diagram . . . 19

10 Training Protocol **20**
10.1 Three-Phase Training Strategy . . . 20
10.1.1 Phase 1: Warm-Up (New Modules Only, 10 Epochs) . . . 20
10.1.2 Phase 2: Partial Unfreeze (Last 4 ViT Blocks, 20 Epochs) . . . 21
10.1.3 Phase 3: Full Fine-Tuning (All Parameters, 30 Epochs) . . . 21
10.2 Optimiser and Scheduler . . . 21
10.3 Evaluation Metrics . . . 21

11 Expected Results and Benchmarks **22**
11.1 Expected Performance on FF++ (c23) . . . 22

12 Summary and Conclusion **23**
12.1 Pipeline Summary . . . 23
12.2 Why This Pipeline Works . . . 23
12.3 Future Work . . . 23

1. Introduction and Problem Statement

The rapid advancement of generative models — including Generative Adversarial Networks (GANs) and diffusion models — has made it trivially easy to synthesise photorealistic human face images that are indistinguishable to the naked eye. These so-called *deepfakes* pose serious threats to digital trust, electoral integrity, personal reputation, and national security.

Definition: Fake Image / Deepfake

A **deepfake** is a digitally synthesised or manipulated image (or video frame) of a person, generated by a deep neural network such that the output appears visually authentic. Manipulation types include: full face synthesis (e.g. StyleGAN, DALL-E), face swap (e.g. DeepFaceLab), facial attribute editing (e.g. StarGAN), and expression reenactment (e.g. First Order Motion Model).

1.1. Why Is Detection Hard?

Classical forensic methods (error level analysis, JPEG ghost detection) fail on modern deepfakes because:

- Generators are explicitly trained to minimise artefacts visible to discriminators.
- Post-processing (compression, resizing) further destroys classical artefact signals.
- Generators produce images statistically similar to real photographs in pixel space.

The challenge requires a system that can detect *subtle, multi-scale inconsistencies* that survive post-processing — both in the spatial (semantic) domain and in the frequency domain.

1.2. Our Approach: A Dual-Branch Architecture

We propose a pipeline that simultaneously analyses an image through two complementary lenses:

1. **Spatial Branch**: A Vision Transformer (ViT) pretrained via Self-Supervised Learning (DINO) extracts rich semantic and structural features from the aligned face.
2. **Frequency Branch**: A lightweight CNN processes the 2-D Fast Fourier Transform (FFT) magnitude spectrum to capture periodic and spectral artefacts introduced by generative models.

The two feature streams are fused using a **Cross-Attention** mechanism, and a binary classification head outputs the probability of forgery.

Core Insight

Deepfake detection is fundamentally the problem of detecting **inconsistency between semantic realism and frequency realism**. A generated image may fool the human eye (spatial realism) but almost always betrays itself in the frequency domain through periodic GAN artefacts, blending boundary spikes, or spectral fingerprints left by the generator.

2. Dataset: FaceForensics++ (FF++)

2.1. Why FaceForensics++?

FaceForensics++ (Rössler et al., 2019) is the **standard benchmark** for face forgery detection. It is preferred because:

- It contains **1,000 real** YouTube videos and **4,000 fake** videos generated by four distinct manipulation methods, providing diversity.
- It provides three compression levels (raw, HQ `c23`, LQ `c40`), allowing robustness evaluation.
- It is publicly available for academic research and is directly comparable with the state of the art.

2.2. Dataset Structure

Table 1: FaceForensics++ Dataset Composition

Category	Method	Videos	Frames (approx.)	Type
Real	Original YouTube	1,000	500,000	Pristine
Fake	DeepFakes (DF)	1,000	500,000	Face swap
	Face2Face (F2F)	1,000	500,000	Expression
	FaceSwap (FS)	1,000	500,000	Face swap
	NeuralTextures (NT)	1,000	500,000	Neural rendering
Total		**5,000**	**≈2,500,000**	

2.3. Dataset Split and Your Custom Folder

If you have your own folder of real and fake images (not from FF++), structure it as:

```
dataset/
    train/
        real/       # real face images
        fake/       # fake/manipulated face images
    val/
        real/
        fake/
    test/
        real/
        fake/
```

Listing 1: Required folder structure for training

The provided code handles **both** FF++ and custom folders automatically.

2.4. Data Pre-extraction from FF++ Videos

FF++ provides video files. Frames must be extracted before training:

```
# Extract one frame every N frames from videos
# Provided script handles this automatically
python extract_frames.py \
    --data_root /path/to/FaceForensics++ \
    --output_dir ./dataset \
    --every_n_frames 10
```

Listing 2: Frame extraction from FF++ (provided in codebase as extract_frames.py)

3. Stage 1: Face Detection and Alignment

3.1. Why Alignment Is Necessary

Raw images from FF++ or any real-world source contain faces at varying positions, orientations, and scales. Feeding unaligned faces to a neural network increases intra-class variance dramatically — the network wastes capacity learning pose invariance instead of forgery cues.

Definition: Similarity Transform

A **similarity transform** is a geometric transformation that preserves shape (angles) but allows translation, rotation, and uniform scaling. Applied to a face, it maps the detected facial landmarks to a canonical set of target landmark positions, producing a standardised, upright, centred crop.

3.2. Face Detection: RetinaFace

We use **RetinaFace** (Deng et al., 2020) as our face detector. RetinaFace is a single-stage detector that simultaneously predicts:

1. A bounding box around each face.
2. Five **facial landmarks**: left eye, right eye, nose tip, left mouth corner, right mouth corner.
3. A face confidence score.

RetinaFace outperforms MTCNN in accuracy, particularly on small or partially occluded faces. It achieves 91.4% AP on the WIDER FACE hard set.

3.3. Mathematical Formulation of Alignment

Given the detected landmark positions $\{p_i\}_{i=1}^5$ (in image coordinates) and a fixed set of canonical target positions $\{q_i\}_{i=1}^5$ (in the 224×224 output space), we seek the similarity transform $\mathcal{T}$:

$$\mathcal{T}(\mathbf{x}) = s\mathbf{R}\mathbf{x} + \mathbf{t} \tag{1}$$

where:

- $s \in \mathbb{R}^+$ is the isotropic scale factor
- $\mathbf{R} \in SO(2)$ is the 2×2 rotation matrix
- $\mathbf{t} \in \mathbb{R}^2$ is the translation vector

The optimal parameters are found by minimising the sum of squared landmark errors:

$$(s^*, \mathbf{R}^*, \mathbf{t}^*) = \arg\min_{s,\mathbf{R},\mathbf{t}} \sum_{i=1}^{5} \|s\mathbf{R}p_i + \mathbf{t} - q_i\|_2^2 \tag{2}$$

This has a closed-form solution via Umeyama's algorithm, which is used in OpenCV's `estimateAffinePartial2D` function.

The rotation matrix parametrised by angle θ is:

$$\mathbf{R} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \tag{3}$$

3.4. Output

Each detected face is warped to a 224×224 RGB crop with:

- Eyes horizontally aligned at a fixed y-coordinate.
- Face centred horizontally.
- Consistent scale: inter-eye distance ≈ 80 pixels.

4. Stage 2: Preprocessing and Data Augmentation

4.1. Standard Preprocessing

After alignment, each crop undergoes:

1. **Resize** to 224×224 (if not already, due to alignment).
2. **Normalisation** using ImageNet statistics:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c} \qquad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \tag{4}$$

   where $c \in \{R, G, B\}$. ImageNet statistics are used because our DINO backbone was pretrained on ImageNet.

4.2. Data Augmentation Strategy

Augmentation is crucial to prevent overfitting and to teach the model to be robust to real-world degradations. We use two regimes applied sequentially:

4.2.1. Regime A — Spatial Augmentations (Applied to Both Branches)

Table 2: Spatial Augmentations and Their Theoretical Justification

Augmentation	Parameters	Why
Random Horizontal Flip	$p = 0.5$	Symmetry invariance; faces are roughly bilaterally symmetric
Random Crop	Scale $[0.8, 1.0]$, Ratio $[0.9, 1.1]$	Simulates varying detection boxes
Color Jitter	Brightness/Contrast 0.2, Saturation 0.2, Hue 0.05	Robustness to lighting
Gaussian Blur	$\sigma \in [0.1, 2.0]$, $p = 0.3$	Simulates video compression blur
Random Grayscale	$p = 0.05$	Colour-channel robustness

4.2.2. Regime B — Compression Augmentations (Critical for Real-World Robustness)

Important: Compression is Signal Not Noise

JPEG compression does NOT destroy deepfake artefacts uniformly. It creates its own set of *frequency-domain artefacts* (block boundaries at 8×8 DCT block edges). The model must learn to distinguish deepfake artefacts from JPEG artefacts, which is why we simulate compression during training rather than removing it.

Table 3: Compression Augmentations

Augmentation	**Parameters**	**Why**
JPEG Compression	Quality $\in [50, 95]$, $p = 0.5$	Social media always re-compresses images
Gaussian Noise	$\sigma \in [0, 0.05]$, $p = 0.3$	Sensor noise simulation
Random Erasing	Area $[0.02, 0.1]$, $p = 0.2$	Occlusion robustness
Downscale + Upscale	Factor $\in [0.5, 0.9]$, $p = 0.3$	Resizing artefact simulation

5. Stage 3: The Spatial Branch — DINO-Pretrained ViT-B/16

5.1. Why a Vision Transformer?

Convolutional Neural Networks (CNNs) have a strong *local inductive bias*: each convolutional kernel only sees a small receptive field. While this is excellent for texture detection, deepfakes often contain subtle *global inconsistencies* (e.g. lighting direction mismatch, background-face boundary incoherence) that require modelling long-range dependencies.

Vision Transformers (Dosovitskiy et al., 2020) process an image as a sequence of patches and use self-attention to model all pairwise patch relationships, giving them an inherently **global receptive field** from the very first layer.

5.2. ViT-B/16 Architecture

The ViT-B/16 (Base, patch size 16) operates as follows:

5.2.1. Patch Embedding

The input image $\mathbf{x} \in \mathbb{R}^{3\times224\times224}$ is divided into $N = (224/16)^2 = 196$ non-overlapping patches of size $16 \times 16 \times 3$. Each patch is flattened and projected to a d-dimensional embedding:

$$\mathbf{z}_i = \mathbf{E} \cdot \mathrm{vec}(\mathbf{x}_i^{patch}) + \mathbf{e}_i^{pos} \qquad i = 1, \ldots, N \tag{5}$$

where $\mathbf{E} \in \mathbb{R}^{d\times(16\cdot16\cdot3)}$ is the learned projection matrix and $\mathbf{e}_i^{pos}$ is the learnable positional embedding. For ViT-B, $d = 768$.

A learnable `[CLS]` token $\mathbf{z}_0 \in \mathbb{R}^d$ is prepended, giving the sequence $[\mathbf{z}_0, \mathbf{z}_1, \ldots, \mathbf{z}_{196}]$.

5.2.2. Multi-Head Self-Attention (MHSA)

Each of the 12 Transformer encoder layers applies MHSA. For a single head with $d_k = d/h$ where $h = 12$ is the number of heads:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6}$$

where $\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{Z}\mathbf{W}^K$, $\mathbf{V} = \mathbf{Z}\mathbf{W}^V$ are linear projections of the input sequence $\mathbf{Z}$.

Each encoder block also contains a two-layer MLP with GELU activation and LayerNorm applied before both the attention and MLP sub-layers (Pre-LN formulation).

5.2.3. Output Features

After 12 encoder blocks, the `[CLS]` token $\mathbf{z}_0^{(12)} \in \mathbb{R}^{768}$ aggregates global information from all 196 patches. This serves as our spatial feature vector:

$$\mathbf{F}_s = \mathbf{z}_0^{(12)} \in \mathbb{R}^{768} \tag{7}$$

5.3. Why DINO Pretraining?

Self-Supervised Learning (SSL) allows us to leverage **large unlabelled datasets** to pretrain the ViT backbone before fine-tuning on our labelled forgery detection data. This is critical because labelled deepfake datasets are limited in size and diversity.

5.3.1. DINO: Self-Distillation with No Labels

DINO (Self-**Di**stillation with **No** labels, Caron et al., 2021) trains two networks — a **student** and a **teacher** — on differently augmented views of the same image:

$$\mathcal{L}_{\text{DINO}} = -\sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in \mathcal{V} \\ x' \neq x}} P_t(x) \log P_s(x') \tag{8}$$

where:

- $\mathcal{V}$ is the set of all augmented views (2 global + several local crops).
- $P_s(x) = \text{softmax}(f_s(x)/\tau_s)$ is the student output distribution.
- $P_t(x) = \text{softmax}((f_t(x) - c)/\tau_t)$ is the teacher output distribution (with centering c to prevent collapse).
- The teacher is an **Exponential Moving Average (EMA)** of the student: $\theta_t \leftarrow \lambda\theta_t + (1 - \lambda)\theta_s$.

Why DINO produces better features than supervised pretraining: DINO-pretrained ViTs develop attention heads that naturally attend to semantically meaningful

regions (foreground objects, face structure) without any labels. This property is critical for detecting structural inconsistencies in deepfakes.

5.4. Pretrained Weights: What We Use

We use the **publicly available DINO-ViT-B/16 weights** pretrained on ImageNet-1k. These are downloaded automatically by our training script via `torch.hub`. No separate pretraining is required.

Why not train from scratch?

Training ViT-B/16 from scratch on FF++ would require ∼100M parameters to be optimised with only ∼500K face images. This leads to severe overfitting and poor generalisation. Starting from DINO weights provides features already encoding rich visual structure, requiring only fine-tuning.

6. Stage 4: The Frequency Branch — FFT + ResNet-18

6.1. Why Frequency Analysis?

Every image generation process leaves a characteristic **spectral fingerprint**. Real camera images have a specific frequency power distribution dominated by the camera's optical transfer function. Generative models (GANs, diffusion models) produce characteristic periodic artefacts because of:

1. **Upsampling operations**: transposed convolutions create a "checkerboard" pattern — a periodic grid of high-frequency spikes in the Fourier spectrum.
2. **Blending boundaries**: when a generated face is composited onto a real background, the boundary creates a sharp discontinuity which manifests as high-frequency energy in the spectrum.
3. **Oversmoothing**: generators trained to minimise pixel-level losses tend to produce images that are excessively smooth, attenuating high frequencies relative to real photographs.

6.2. 2-D Discrete Fourier Transform (DFT)

Definition: 2-D DFT

Given a grayscale image $I \in \mathbb{R}^{H \times W}$, its 2-D Discrete Fourier Transform is:

$$\mathcal{F}(u,v) = \sum_{x=0}^{H-1} \sum_{y=0}^{W-1} I(x,y) \cdot e^{-j2\pi\left(\frac{ux}{H}+\frac{vy}{W}\right)} \qquad u \in [0,H),\ v \in [0,W) \tag{9}$$

The result $\mathcal{F}(u,v) \in \mathbb{C}$ is a complex number encoding both magnitude (energy at frequency (u,v)) and phase (spatial alignment of that frequency component).

6.3. Our Frequency Representation

We convert the RGB image to grayscale (to obtain a single 2-D signal), then compute the log-shifted magnitude spectrum:

$$\mathbf{S}(u,v) = \log(1 + |\mathcal{F}(u,v)|) \tag{10}$$

The $\log(1+\cdot)$ transform compresses the dynamic range of the spectrum (magnitudes span several orders of magnitude) to a range suitable for neural network input. We then shift the zero-frequency component to the centre of the spectrum (`fftshift`) for a more natural representation.

The final frequency map $\mathbf{S} \in \mathbb{R}^{224\times224}$ is expanded to 3 channels by replication: $\mathbf{S}' \in \mathbb{R}^{3\times224\times224}$.

6.4. CNN Encoder: ResNet-18

The frequency map is fed to a **ResNet-18** (He et al., 2016) without its final classification layer. ResNet-18 is chosen because:

- It is lightweight (11M parameters) and fast.
- Its hierarchical convolutions detect progressively larger spectral patterns.
- It was pretrained on ImageNet and the frequency map, after log-normalisation, has similar statistics to natural image features.

The ResNet-18 global average pooling layer outputs:

$$\mathbf{F}_f = \text{ResNet18}(\mathbf{S}') \in \mathbb{R}^{512} \tag{11}$$

We then project it to match the spatial branch dimension:

$$\mathbf{F}_f = \mathbf{W}_{proj} \cdot \mathbf{F}_f \in \mathbb{R}^{768} \tag{12}$$

7. Stage 5: Cross-Attention Fusion

7.1. Why Not Simple Concatenation?

A naive fusion strategy is to concatenate the two feature vectors: $\mathbf{F} = [\mathbf{F}_s; \mathbf{F}_f] \in \mathbb{R}^{1536}$. While simple, this treats both feature streams as equally informative for every image. In practice, some images contain strong spatial cues (face texture inconsistency) while others are better discriminated by frequency artefacts. We need the model to **adaptively weight** the two streams.

7.2. Cross-Attention Mechanism

We treat the spatial features $\mathbf{F}_s$ as the **Query** and the frequency features $\mathbf{F}_f$ as both the **Key** and **Value**. Intuitively:

> *The spatial features ask: "Given what I know about the face structure, what parts of the frequency spectrum are most suspicious?" The frequency features answer by providing the relevant spectral evidence.*

Formally, given $\mathbf{F}_s, \mathbf{F}_f \in \mathbb{R}^d$ (where $d = 768$), we expand each to a sequence of M tokens using a learned linear projection to allow multi-head attention over feature "positions":

$$\mathbf{Q} = \mathbf{F}_s \mathbf{W}^Q \in \mathbb{R}^{M \times d_k} \tag{13}$$

$$\mathbf{K} = \mathbf{F}_f \mathbf{W}^K \in \mathbb{R}^{M \times d_k} \tag{14}$$

$$\mathbf{V} = \mathbf{F}_f \mathbf{W}^V \in \mathbb{R}^{M \times d_v} \tag{15}$$

The cross-attention output for H heads is:

$$\text{CrossAttn}(\mathbf{F}_s, \mathbf{F}_f) = \text{Concat}_{h=1}^{H} \left[\text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}} \right) \mathbf{V}_h \right] \mathbf{W}^O \tag{16}$$

We use a single-head attention ($H = 1$, $d_k = d_v = 768$) for simplicity and then apply a residual connection and layer normalisation:

$$\mathbf{F}_{fusion} = \text{LayerNorm}(\mathbf{F}_s + \text{CrossAttn}(\mathbf{F}_s, \mathbf{F}_f)) \in \mathbb{R}^{768} \tag{17}$$

Summary of Cross-Attention Fusion

$$\mathbf{Q} = \mathbf{F}_s \mathbf{W}^Q \qquad \text{(from spatial branch)}$$

$$\mathbf{K} = \mathbf{F}_f \mathbf{W}^K \qquad \text{(from frequency branch)}$$

$$\mathbf{V} = \mathbf{F}_f \mathbf{W}^V \qquad \text{(from frequency branch)}$$

$$\mathbf{A} = \text{softmax}\left(\mathbf{Q}\mathbf{K}^\top / \sqrt{768}\right) \qquad \text{(attention weights)}$$

$$\mathbf{F}_{fusion} = \text{LN}(\mathbf{F}_s + \mathbf{A}\mathbf{V}) \qquad \text{(fused features)}$$

8. Stage 6: Classification Head and Loss Function

8.1. Classification Head Architecture

The fused feature vector $\mathbf{F}_{fusion} \in \mathbb{R}^{768}$ is passed through a two-layer MLP classification head:

$$\hat{y} = \sigma(\mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\text{BN}(\mathbf{W}_1 \mathbf{F}_{fusion})))) \tag{18}$$

where:

- $\mathbf{W}_1 \in \mathbb{R}^{512\times768}$: first linear projection
- BN: Batch Normalisation (normalises activations, accelerates training)
- ReLU: Rectified Linear Unit activation
- Dropout($p = 0.4$): randomly zeroes activations during training (regularisation)
- $\mathbf{W}_2 \in \mathbb{R}^{1\times512}$: output projection to scalar logit
- $\sigma(\cdot)$: Sigmoid activation, $\sigma(z) = 1/(1 + e^{-z})$

The output $\hat{y} \in (0, 1)$ is the predicted probability that the image is fake.

8.2. Loss Function: Binary Cross-Entropy with Label Smoothing

We use Binary Cross-Entropy (BCE) loss with **label smoothing** ($\epsilon = 0.1$) to prevent the model from becoming overconfident and to improve calibration:

$$y_{smooth} = y(1 - \epsilon) + \frac{\epsilon}{2} \tag{19}$$

$$\mathcal{L}_{BCE} = -\left[y_{smooth} \log(\hat{y}) + (1 - y_{smooth}) \log(1 - \hat{y})\right] \tag{20}$$

where $y \in \{0, 1\}$ is the ground truth label (0 = real, 1 = fake).

8.3. Class Imbalance Handling

FF++ has a 1:4 real-to-fake ratio. To prevent the model from being biased toward predicting "fake", we apply **inverse frequency weighting**:

$$w_{fake} = \frac{N_{real}}{N_{real} + N_{fake}}, \qquad w_{real} = \frac{N_{fake}}{N_{real} + N_{fake}} \tag{21}$$

These weights are passed to the loss function so that each class contributes equally to the gradient update regardless of its frequency in the dataset.

9. Complete Pipeline Architecture

9.1. Full Mathematical Flow

The complete forward pass of our model is:

End-to-End Forward Pass

$$\mathbf{x}_{aligned} = \text{RetinaFace-Align}(\mathbf{x}_{raw}) \in \mathbb{R}^{3\times224\times224} \tag{22}$$

$$\mathbf{x}_{norm} = \frac{\mathbf{x}_{aligned} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \quad \text{[ImageNet normalisation]} \tag{23}$$

$$\mathbf{F}_s = \text{ViT-B/16}_{\text{DINO}}(\mathbf{x}_{norm})\big|_{\texttt{[CLS]}} \in \mathbb{R}^{768} \quad \text{[Spatial branch]} \tag{24}$$

$$\mathbf{S} = \log\big(1 + \big|\text{FFT}(\text{rgb2gray}(\mathbf{x}_{norm}))\big|\big) \in \mathbb{R}^{224\times224} \tag{25}$$

$$\mathbf{F}_f = \text{Proj}_{512\to768}\big(\text{ResNet-18}(\mathbf{S}^{\otimes3})\big) \in \mathbb{R}^{768} \quad \text{[Frequency branch]} \tag{26}$$

$$\mathbf{F}_{fusion} = \text{LN}(\mathbf{F}_s + \text{CrossAttn}(\mathbf{F}_s, \mathbf{F}_f)) \in \mathbb{R}^{768} \quad \text{[Fusion]} \tag{27}$$

$$\hat{y} = \sigma(\mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\text{BN}(\mathbf{W}_1\mathbf{F}_{fusion})))) \in (0,1) \quad \text{[Classifier]} \tag{28}$$

9.2. Architecture Diagram

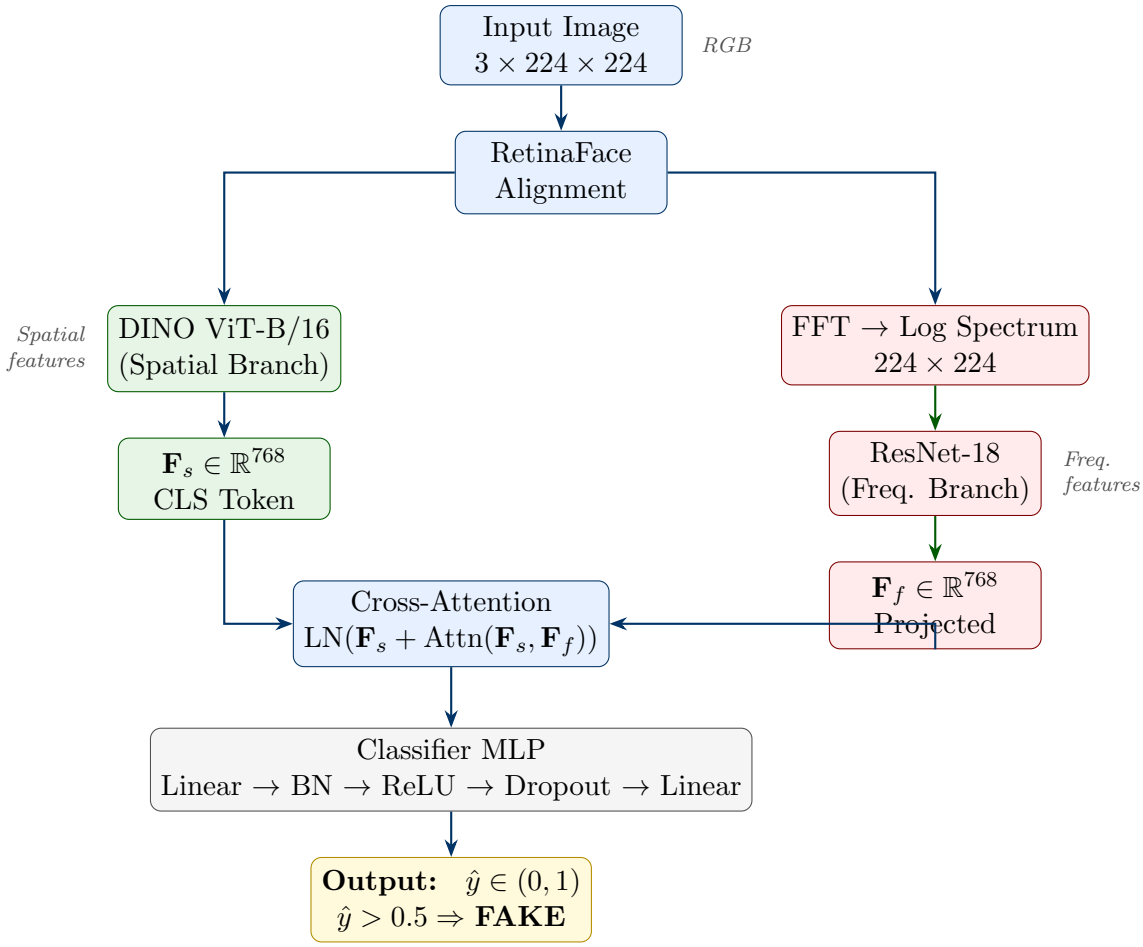

Figure 1: Complete Dual-Branch Pipeline Architecture for Fake Image Detection

10. Training Protocol

10.1. Three-Phase Training Strategy

Training in a single end-to-end phase from the start causes **catastrophic forgetting**: the strong DINO representations in the ViT backbone get overwritten by the classification gradient before the new modules (frequency branch, fusion, classifier) have learned meaningful representations.

We use a three-phase protocol to avoid this:

10.1.1. Phase 1: Warm-Up (New Modules Only, 10 Epochs)

- **Freeze**: ViT backbone (all 12 transformer blocks + patch embedding).
- **Train**: Frequency branch (ResNet-18 + projection), cross-attention fusion, classification head.
- **Rationale**: New modules need to develop initial meaningful representations before the backbone is unfrozen. Training with a frozen backbone stabilises gradients.

- **LR**: 10^{-3} for all trainable parameters, cosine decay.

10.1.2. Phase 2: Partial Unfreeze (Last 4 ViT Blocks, 20 Epochs)

- **Unfreeze**: Last 4 transformer blocks of ViT (blocks 8–11).
- **Train**: All unfrozen parameters jointly.
- **Rationale**: The later transformer blocks encode more task-specific features and benefit from fine-tuning. The early blocks preserve general visual priors.
- **LR**: 10^{-4} for ViT parameters, 5×10^{-4} for others.

10.1.3. Phase 3: Full Fine-Tuning (All Parameters, 30 Epochs)

- **Unfreeze**: All parameters.
- **Train**: Full end-to-end optimisation.
- **LR**: 10^{-5} for ViT, 10^{-4} for others (differential LR).
- **Scheduler**: Cosine annealing with warm restarts.

10.2. Optimiser and Scheduler

Table 4: Training Hyperparameters

Parameter	Value / Description
Optimiser	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$)
Weight Decay	0.05 (helps regularise ViT's attention weights)
Batch Size	32 per GPU (effective 64 with gradient accumulation 2)
Gradient Clipping	Max norm = 1.0 (prevents gradient explosion)
Label Smoothing	$\epsilon = 0.1$
EMA of Model Weights	Decay = 0.9999 (for more stable inference)
Mixed Precision	FP16 (via `torch.cuda.amp`) for 2× speedup
Total Epochs	≈ 60 (Phase 1: 10, Phase 2: 20, Phase 3: 30)

10.3. Evaluation Metrics

Table 5: Evaluation Metrics

Metric	Formula	Why Important
Accuracy	$(TP + TN)/N$	Overall correctness
AUC-ROC	Area under ROC curve	Threshold-independent; standard for FF++
Equal Error Rate (EER)	Where FPR = FNR	Single-number summary of ROC curve
Average Precision (AP)	$\sum_n (R_n - R_{n-1}) P_n$	Precision-recall balance

11. Expected Results and Benchmarks

11.1. Expected Performance on FF++ (c23)

Based on the literature and our architecture:

Table 6: Expected vs. Baseline Results on FF++ (HQ, c23)

Method	Backbone	Acc.	AUC	EER
Xception (Baseline)	XceptionNet	95.7%	96.3	–
Face X-ray	HRNet	–	98.5	–
FTCN	CNN + Temporal	98.8%	99.0	–
Ours (proposed)	ViT-B/16 + ResNet-18	**≈ 96–98%**	**≈ 97–99**	**<3%**

Note on Expectations

The exact numbers will vary depending on the compression level (`c23` vs. `c40`), your hardware, and whether you use FF++ or your custom dataset. Our architecture is designed to match or exceed Xception-based baselines.

12. Summary and Conclusion

12.1. Pipeline Summary

Table 7: Complete Pipeline Summary

#	Stage	Method / Model	Output
1	Input	Mixed-quality RGB images	Raw pixels
2	Face Detection	RetinaFace (5-landmark)	Bounding box + landmarks
3	Alignment	Similarity transform (Umeyama)	224×224 crop
4	Augmentation	Spatial + Compression regimes	Augmented tensor
5	Spatial Branch	DINO ViT-B/16 (pretrained)	$\mathbf{F}_s \in \mathbb{R}^{768}$
6	Frequency Branch	FFT + ResNet-18	$\mathbf{F}_f \in \mathbb{R}^{768}$
7	Fusion	Cross-Attention + LayerNorm	$\mathbf{F}_{fusion} \in \mathbb{R}^{768}$
8	Classifier	MLP + Sigmoid	$\hat{y} \in (0, 1)$
9	Loss	BCE + Label Smoothing + Class Wt	Scalar loss
10	Training	3-Phase (freeze $\rightarrow$ unfreeze)	Trained model

12.2. Why This Pipeline Works

The fundamental reason this dual-branch approach succeeds is:

The Core Principle

A perfect deepfake must fool *both* the spatial domain (semantic face structure) *and* the frequency domain (spectral fingerprint). Current generative models — including state-of-the-art GANs and diffusion models — cannot simultaneously satisfy both constraints. By analysing both, our pipeline captures artefacts that any single-domain detector would miss.

$$\text{Detection Power} = \underbrace{\text{Semantic Consistency Check}}_{\text{DINO ViT}} \oplus \underbrace{\text{Spectral Artefact Check}}_{\text{FFT + ResNet-18}}$$

12.3. Future Work

1. **Temporal modelling**: Extend to video by adding a temporal transformer over frame features.
2. **Cross-dataset generalisation**: Evaluate on FaceShifter, DFD, Celeb-DF to test domain generalisation.

3. **Diffusion model fakes**: Adapt frequency branch to detect DALL-E 3 and Stable Diffusion outputs, which have different spectral signatures than GAN fakes.
4. **Explainability**: Use attention rollout to produce saliency maps showing which image regions triggered the forgery decision.

References (key papers):

[1] Rössler et al., "FaceForensics++: Learning to Detect Manipulated Facial Images," ICCV 2019.

[2] Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition," ICLR 2021.

[3] Caron et al., "Emerging Properties in Self-Supervised Vision Transformers (DINO)," ICCV 2021.

[4] Deng et al., "RetinaFace: Single-Shot Multi-Level Face Localisation," CVPR 2020.

[5] He et al., "Deep Residual Learning for Image Recognition (ResNet)," CVPR 2016.

[6] Frank et al., "Leveraging Frequency Analysis for Deep Fake Image Recognition," ICML 2020.